Console output

```
We Have The Following Employeers
Engineer: Steve Wozniak, ID: 101, Specialty: Hardware, Base Salary: 50000,
        Total Salary: 50000
Engineer: Linus Torvalds, ID: 103, Specialty: Software, Base Salary: 45000,
        Total Salary: 45000
Engineer: Elon Musk, ID: 104, Specialty: Spacecraft Engineering, Base Salary:
        47000, Total Salary: 47000
Engineer: Alan Turing, ID: 105, Specialty: Cryptography, Base Salary: 48000,
        Total Salary: 48000
Engineer: Ada Lovelace, ID: 106, Specialty: Mathematics, Base Salary: 49000,
        Total Salary: 49000
Engineer: Grace Hopper, ID: 107, Specialty: Computer Programming, Base Salary:
        52000, Total Salary: 52000
Manager: Steve Jobs, ID: 102, Team Size: 5, Base Salary: 70000, Total Salary:
        70500
Manager: Bill Gates, ID: 108, Team Size: 4, Base Salary: 65000, Total Salary:
        65400
Manager: Sheryl Sandberg, ID: 109, Team Size: 6, Base Salary: 68000, Total
        Salary: 68600
Manager: Jeff Bezos, ID: 110, Team Size: 3, Base Salary: 72000, Total Salary:
        72300

After Promotion:
Engineer: Steve Wozniak, ID: 101, Specialty: Hardware, Base Salary: 57500,
        Total Salary: 57500
Engineer: Linus Torvalds, ID: 103, Specialty: Software, Base Salary: 51750,
        Total Salary: 51750
Engineer: Elon Musk, ID: 104, Specialty: Spacecraft Engineering, Base Salary:
        54050, Total Salary: 54050
Engineer: Alan Turing, ID: 105, Specialty: Cryptography, Base Salary: 55200,
        Total Salary: 55200
Engineer: Ada Lovelace, ID: 106, Specialty: Mathematics, Base Salary: 56350,
        Total Salary: 56350
Engineer: Grace Hopper, ID: 107, Specialty: Computer Programming, Base Salary:
        59800, Total Salary: 59800
Manager: Steve Jobs, ID: 102, Team Size: 5, Base Salary: 84000, Total Salary:
        84500
Manager: Bill Gates, ID: 108, Team Size: 4, Base Salary: 78000, Total Salary:
        78400
Manager: Sheryl Sandberg, ID: 109, Team Size: 6, Base Salary: 81600, Total
        Salary: 82200
Manager: Jeff Bezos, ID: 110, Team Size: 3, Base Salary: 86400, Total Salary:
        86700

After Sorting By Salary:
Manager: Jeff Bezos, ID: 110, Team Size: 3, Base Salary: 86400, Total Salary:
        86700
Manager: Steve Jobs, ID: 102, Team Size: 5, Base Salary: 84000, Total Salary:
        84500
Manager: Sheryl Sandberg, ID: 109, Team Size: 6, Base Salary: 81600, Total
        Salary: 82200
Manager: Bill Gates, ID: 108, Team Size: 4, Base Salary: 78000, Total Salary:
        78400
Engineer: Grace Hopper, ID: 107, Specialty: Computer Programming, Base Salary:
        59800, Total Salary: 59800
Engineer: Steve Wozniak, ID: 101, Specialty: Hardware, Base Salary: 57500,
        Total Salary: 57500
Engineer: Ada Lovelace, ID: 106, Specialty: Mathematics, Base Salary: 56350,
        Total Salary: 56350
Engineer: Alan Turing, ID: 105, Specialty: Cryptography, Base Salary: 55200,
        Total Salary: 55200
Engineer: Elon Musk, ID: 104, Specialty: Spacecraft Engineering, Base Salary:
        54050, Total Salary: 54050
Engineer: Linus Torvalds, ID: 103, Specialty: Software, Base Salary: 51750,
        Total Salary: 51750
```

employee.cpp

```
/**
 * @file employee.cpp
 * @brief Employee Base Class.
 * @author Jorge Louro
 * @bug No Bugs
 */

#include "employee.h"
#include <iostream>
#include <algorithm>

/**
 * @brief Employee object
 *
 * @param name Employee Name
 * @param id Employee ID
 * @param baseSalary Employee Base Salary
 */
Employee::Employee(std::string name, int id, double baseSalary)
```

employee.cpp (cont'd)

```
        : name(std::move(name)), id(id), baseSalary(baseSalary) {}

/**
 * @brief Virtual Destructor For Employee
 */
Employee::~Employee() {}

/**
 * @brief Calculate The Total Salary Of The Employee
 *
 * @return Double Total Salary
 */
double Employee::calculateSalary() const {
    return baseSalary;
}

/**
 * @brief Promote The Employee
 */
void Employee::promote() {

}

/**
 * @brief Output Stream Operator For Employee
 *
 * @param os Output Stream
 * @param emp Employee Object
 * @return std::ostream& Output Stream With Employee Details
 */
std::ostream& operator<<(std::ostream& os, const Employee& emp) {
    emp.display();
    return os;
}

/**
 * @brief Sort Employees By Their Total Salary (In Descending Order)
 *
 * @param employees Vector Of Pointers To Employee Objects
 */
void sortEmployeesBySalary(std::vector<Employee*>& employees) {
    std::sort(employees.begin(), employees.end(), [](Employee* a, Employee* b)
        {
        return a->calculateSalary() > b->calculateSalary();
    });
}
```

engineer.cpp

```
/**
 * @file engineer.cpp
 * @brief Engineer Derived Class.
 * @author Jorge Louro
 * @bug No Bugs
 */

#include "engineer.h"
#include <iostream>

/**
 * @brief Engineer Object
 *
 * @param name Engineer Name
 * @param id Engineer ID
 * @param baseSalary Engineer Base Salary
 * @param specialty Engineer Specialty
 */
Engineer::Engineer(std::string name, int id, double baseSalary, std::string
        specialty)
        : Employee(std::move(name), id, baseSalary),
        specialty(std::move(specialty)) {}

/**
 * @brief Display The Engineer Details
 */
void Engineer::display() const {
    std::cout << "Engineer: " << name << ", ID: " << id << ", Specialty: " <<
        specialty
              << ", Base Salary: " << baseSalary << ", Total Salary: " <<
        calculateSalary();
}

/**
 * @brief Get The Type Of Employee
```

engineer.cpp (cont'd)

```
 *
 * @return std::string Type Of Employee
 */
std::string Engineer::getType() const {
    return "Engineer";
}

/**
 * @brief Calculate The Total Salary Of The Engineer
 *
 * @return Double Total salary
 */
double Engineer::calculateSalary() const {
    return baseSalary;
}

/**
 * @brief Promote The Engineer
 */
void Engineer::promote() {
    baseSalary *= 1.15;
}
```

manager.cpp

```
/**
 * @file manager.cpp
 * @brief Implementation of the Manager derived class.
 * @author Jorge Louro
 * @bug No Bugs
 */

#include "manager.h"
#include <iostream>

/**
 * @brief Manager Object
 *
 * @param name Manager Name
 * @param id Manager ID
 * @param baseSalary Manager Base Salary
 * @param teamSize Size Of The Manager's Team
 */
Manager::Manager(std::string name, int id, double baseSalary, int teamSize)
        : Employee(std::move(name), id, baseSalary), teamSize(teamSize) {}

/**
 * @brief Display the manager details
 */
void Manager::display() const {
    std::cout << "Manager: " << name << ", ID: " << id << ", Team Size: " <<
        teamSize
              << ", Base Salary: " << baseSalary << ", Total Salary: " <<
        calculateSalary();
}

/**
 * @brief Get The Type Of Employee
 *
 * @return std::string Type of employee
 */
std::string Manager::getType() const {
    return "Manager";
}

/**
 * @brief Calculate The Total Salary Of The Manager
 *
 * @return Double Total salary
 */
double Manager::calculateSalary() const {
    return baseSalary + 100 * teamSize;
}

/**
 * @brief Promote The Manager
 */
void Manager::promote() {
    baseSalary *= 1.20;
}
```